SIMATS ENGINEERING
Engineer to Excel

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Course Code: CSA1741 Course Name: ARTIFICIAL INTELLIGENCE

Slot: B

ASSIGNMENT 2 – CO2 (ANSWERS)

Nature of Assignment: Individual Total Marks: 50

Student Name: TANUJA P Reg. No: 192321052

Note on Experimental Evidence

This report provides the algorithm designs, mathematical reasoning, pseudocode and Python prototypes. The assignment requires measured operation counts and raw outputs from the student's own execution. Therefore, numerical experiment results are not presented as if they were measured on your computer. The code includes counters and test cases so that genuine results can be generated and inserted before submission.

TASK 1 – Greedy Best-First and A* Under Different Map Conditions (6 Marks)

1.1 Prediction for Different Map Conditions

Uniform-Cost Search (UCS) expands the lowest-cost path first. It is complete and optimal when step costs are positive, but it does not use distance-to-goal information. Greedy Best-First Search (GBFS) expands the node with the smallest heuristic value h(n). It can be fast when the heuristic points directly toward the goal, but it can be misled by obstacles and dead ends and is not generally optimal. A* expands the node with the smallest f(n)=g(n)+h(n). With an admissible heuristic, A* finds an optimal path under the usual graph-search conditions.

Map condition	UCS	Greedy best-first	A*
Open map	May expand many cells by increasing path cost.	Usually moves directly toward the goal.	Usually efficient; heuristic guides search while preserving optimality.
Cluttered map	Reliable but may explore many low-cost alternatives.	May be diverted by obstacles and detours.	Balances progress and actual cost; generally robust.
Maze-like map	Can explore a large part of the maze before reaching the goal.	Can follow a misleading direction and backtrack.	Uses heuristic plus accumulated cost; may still expand many nodes.
Dead-end trap	Eventually finds a route if one exists.	Can enter the trap because it looks close to the goal.	Usually avoids expensive detours

			when g(n) exposes their cost.

1.2 Admissible Heuristic

For a four-direction grid in which each move costs 1, use Manhattan distance: $h(n)=|x_n-x_{goal}|+|y_n-y_{goal}|$. The relaxed problem removes all obstacles, so the shortest possible route is the Manhattan distance. Obstacles can only keep the real route the same length or make it longer. Therefore h(n) never overestimates the true remaining cost and is admissible. It is also consistent for unit-cost four-direction movement.

1.3 Three Designed Maps

- Map A – Open: a 10×10 grid with start (0,0), goal (9,9), and no obstacles. This checks whether the heuristic efficiently directs the search.
- Map B – Cluttered/maze: a 10×10 grid with a vertical wall and one opening near the bottom. This forces a detour and tests whether GBFS is misled.
- Map C – Dead-end edge case: place a long corridor toward the goal that ends at a blocked cell, while a longer-looking side route reaches the goal. This tests greedy commitment to a misleading direction.

1.4 Pseudocode

```
UCS(start, goal):
  frontier = priority queue containing (0, start)
  best_cost[start] = 0
  while frontier is not empty:
    remove node n with smallest g
    if n is goal: return reconstructed path
    for each valid neighbour v:
      new_cost = g[n] + step_cost(n,v)
      if v is unseen or new_cost < best_cost[v]:
        update best_cost[v], parent[v]
        insert (new_cost, v)
  return failure

GBFS(start, goal):
  frontier = priority queue containing (h(start), start)
  while frontier is not empty:
    remove node n with smallest h(n)
    if n is goal: return reconstructed path
    add each unvisited valid neighbour v with priority h(v)
  return failure

A*(start, goal):
  frontier = priority queue containing (h(start), start)
  g[start] = 0
  while frontier is not empty:
```

```
  remove node n with smallest g[n] + h(n)
  if n is goal: return reconstructed path
  for each valid neighbour v:
    candidate = g[n] + step_cost(n,v)
    if candidate < g.get(v, infinity):
      g[v] = candidate; parent[v] = n
      insert/update v with priority g[v] + h(v)
return failure
```

1.5 Recommendation for 95% Open-Road Trips

Recommend A* with Manhattan distance for the shuttle. It uses the strong directional information available on mostly open grid roads while retaining optimality for unit-cost movement. GBFS may be faster in some open cases but can return a non-optimal route in detour cases. UCS is a safe baseline but wastes work because it ignores the goal direction. If the road network is extremely simple and only a feasible route is needed, GBFS may be considered, but A* is the better general default.

TASK 2 – When Is an Online Search Agent an Appropriate Engineering Choice? (4 Marks)

2.1 Experimental Design

An online search agent does not know the complete map. It senses neighbouring cells, moves, and records what it discovers. Online depth-first search (DFS) is suitable when the environment is initially unknown and exploration is necessary. It can be evaluated with a small, medium and large maze, each containing a low and high dead-end density.

Test	Environment	Target	Purpose
1	Small open grid	Near start	Baseline movement cost
2	Medium maze	Far from start	Effect of distance and obstacles
3	Large maze, many dead ends	Reachable target	Revisits and exploration overhead
4	Large disconnected maze	Unreachable target	Termination and failure detection

2.2 Measures and Interpretation

- Moves: total physical steps made by the rover, including backtracking.
- Revisited cells: number of times a previously visited cell is entered again.
- Optimal cost: shortest-path distance calculated after the full map is revealed.
- Competitive ratio: travelled cost ÷ optimal cost. It is meaningful for reachable targets; for an unreachable target, report failure and exploration coverage instead of dividing by zero.
- Success rate: successful target discoveries divided by total trials.

2.3 Operating Boundary

Online DFS is reasonable when the map is unknown, sensing is local, the facility is small or moderate, and exploration itself is useful. It is less suitable when the facility is very large, dead ends are frequent,

time or battery is limited, or the same route must be travelled repeatedly. For an unreachable target, the agent may need to explore the entire reachable component before proving that no route exists.

2.4 Changed Condition: 20 Rovers, 20 Stable Facilities

When each facility has a stable map for months, repeatedly exploring from scratch is wasteful. The first rover can build a map and share it with the other rovers, subject to safety and update checks. Later missions can use an offline route planner. LRTA* can improve decisions across repeated trials, but shared mapping and reuse of known routes are more appropriate when the maps remain unchanged. Online exploration should be retained only for unexpected changes or newly discovered blocked passages.

TASK 3 – Engineer the Behaviour of Alpha-Beta Pruning (6 Marks)

3.1 Principle

Minimax evaluates a game tree by alternating MAX and MIN levels. Alpha-beta pruning gives the same final minimax decision but avoids exploring branches that cannot affect the result. Alpha is the best value MAX can guarantee so far; beta is the best value MIN can guarantee so far. A branch can be pruned when alpha is greater than or equal to beta.

3.2 Designed Test Trees and Predictions

- Tree A – Good move ordering: place the strongest MAX option first at MAX nodes and the weakest MIN reply first at MIN nodes. Alpha-beta should produce many cutoffs and visit far fewer leaves than minimax.
- Tree B – Poor move ordering: arrange the least useful options first so that bounds are discovered late. Alpha-beta should approach minimax's full node count.
- Tree C – Equal leaf values or shuffled move order: test whether equal values and ordering affect the number of cutoffs. The final minimax value should remain unchanged.

For branching factor b and depth d, minimax visits O(b^d) nodes. With ideal ordering, alpha-beta can reduce the effective search to approximately O(b^(d/2)); with poor ordering, its work can approach O(b^d). The exact count depends on tree shape, leaf values and move order.

3.3 Pseudocode

```
MINIMAX(node, depth, maximizing):
  if depth = 0 or node is terminal: return evaluate(node)
  if maximizing:
    value = -infinity
    for child in children(node):
      value = max(value, MINIMAX(child, depth-1, false))
    return value
  else:
    value = +infinity
    for child in children(node):
      value = min(value, MINIMAX(child, depth-1, true))
```

```
    return value

ALPHA_BETA(node, depth, alpha, beta, maximizing):
  if depth = 0 or node is terminal: return evaluate(node)
  if maximizing:
    value = -infinity
    for child in ordered_children(node):
      value = max(value, ALPHA_BETA(child, depth-1, alpha, beta, false))
      alpha = max(alpha, value)
      if alpha >= beta: record cutoff; break
    return value
  else:
    value = +infinity
    for child in ordered_children(node):
      value = min(value, ALPHA_BETA(child, depth-1, alpha, beta, true))
      beta = min(beta, value)
      if beta <= alpha: record cutoff; break
    return value
```

3.4 Analysis

A tree with good move ordering requires less work because a strong bound is found early and later branches are proven irrelevant. Poor ordering delays useful bounds, so fewer branches can be discarded. Branching factor and depth alone are not enough to predict alpha-beta performance; the order and values of moves also matter. The minimax value must be identical with and without pruning.

TASK 4 – Minimax and the Cost of Examining Every Possibility (6 Marks)

4.1 Depth-Limited Minimax

At each turn, the game tree branches into the legal moves. MAX chooses the largest child value and MIN chooses the smallest child value. When the depth limit is reached, the evaluation function estimates how good the position is for MAX.

4.2 Example Evaluation Function

For a simple strategy game, define: $E(s)=3\times(\text{MAX pieces}-\text{MIN pieces})+2\times(\text{MAX territory}-\text{MIN territory})+5\times(\text{MAX objectives}-\text{MIN objectives})$. Positive values favour MAX; negative values favour MIN. The weights are design choices and should be tested against game outcomes.

4.3 Mathematical Node Count

For a full uniform tree with branching factor b and search depth d, the number of nodes including the root is $N=1+b+b^2+\ldots+b^d=(b^{(d+1)}-1)/(b-1)$, for $b\neq1$. The number of leaf evaluations at exactly depth d is b^d. If the tree ends earlier because of terminal positions, the actual count is smaller.

Branching factor b	Depth d	Expected leaves b^d
3	2	9
3	4	81

4	4	256
5	5	3125

4.4 Scalability

The time complexity is O(b^d), while recursive stack space is O(bd) for depth-first minimax. Increasing depth by one multiplies the leaf count by approximately b. A response-time limit should be selected for the actual application; for this prototype, use 1 second as an example target and record the machine and test conditions. Plain minimax becomes impractical when measured response time exceeds the target or when the required depth cannot be completed before the move deadline.

The main inefficiency is examining every legal continuation, including branches that cannot influence the final choice. A redesigned engine should reduce unnecessary search through pruning, better move ordering, reuse of previously evaluated positions and/or a more selective search. These methods trade memory or implementation complexity for fewer evaluations.

TASK 5 – Derive a Constraint-Propagation Algorithm from a Principle (7 Marks)

5.1 CSP Representation

Represent each exam as a variable X_i. Its domain D_i contains available time slots. If two exams share students, add a binary constraint requiring their slots to be different. The exam timetable is a constraint-satisfaction problem (CSP) consisting of variables, domains and constraints.

5.2 Arc Consistency Rule

For every directed pair (X_i,X_j) connected by a constraint, inspect each value x in D_i. Keep x only if at least one value y in D_j satisfies the constraint. If no supporting y exists, remove x. When a value is removed from D_i, neighbouring arcs that may have lost support must be added back to the queue.

5.3 AC-3 Style Pseudocode

```
PROPAGATE(domains, neighbours):
  queue = all directed arcs (Xi, Xj) for constrained pairs
  while queue is not empty:
    (Xi, Xj) = remove one arc
    if REVISE(Xi, Xj):
      if domain[Xi] is empty: return FAILURE
      for each Xk in neighbours[Xi], where Xk != Xj:
        add (Xk, Xi) to queue
  return SUCCESS

REVISE(Xi, Xj):
  changed = false
  for each value x in a copy of domain[Xi]:
    if no y in domain[Xj] satisfies constraint(Xi=x, Xj=y):
      remove x from domain[Xi]
      changed = true
  return changed
```

```
BACKTRACK(assignment):
  if all variables assigned: return assignment
  choose an unassigned variable
  for each value in its current domain:
    assign value
    if PROPAGATE succeeds:
      result = BACKTRACK(assignment)
      if result succeeds: return result
    undo assignment and domain changes
  return FAILURE
```

5.4 Test Cases and Correctness

- Propagation-only success: three exams A, B and C share students pairwise and each has a distinct singleton slot after initial restrictions. Propagation confirms the assignment without branching.
- Failure case: two exams share students but both have the singleton domain {Slot 1}. Arc revision removes the unsupported value and creates an empty domain, proving that the current CSP has no solution.
- Challenge case: use five exams with overlapping student groups and domains {1,2,3}. Compare ordinary backtracking with backtracking plus propagation.

Removing an unsupported value is safe because any valid solution assigning x to X_i must also assign a compatible value to every constrained neighbour X_j. If no such value exists in D_j, no complete solution can contain x. Therefore, removing x cannot remove a genuine solution. Propagation may not solve every CSP by itself; backtracking is still needed when multiple supported values remain.

Measure search nodes, backtracks, constraint checks, domain wipe-outs and the number of values removed. Record the domain sizes after each propagation round to visualise shrinking domains.

TASK 6 – Exhaustive Search, Local Search and Combinatorial Growth (8 Marks)

6.1 Exhaustive Tour Enumeration

Fix the depot as the first city and generate every permutation of the n customers. Append the depot at the end to form a complete tour. Calculate the sum of edge distances and retain the tour with the smallest cost. For a symmetric travelling-salesperson instance, reversing a tour gives the same cost, so symmetry can be reduced by fixing one customer's order, but the basic exhaustive version can enumerate all $(n-1)!$ tours.

6.2 Complexity

The number of candidate tours is $(n-1)!$ when the depot is fixed. Each tour requires $O(n)$ distance additions, so the total time is $O(n \cdot (n-1)!)$ and the recursion/working space is $O(n)$, excluding stored input distances. The factorial growth makes exhaustive search suitable only for small instances.

Customers n	Tours $(n-1)!$	Growth
5	24	Small

8	5040	Manageable for a simple program
10	362880	Much larger
12	39916800	Expensive

6.3 Correctness Check

Use a four-city example with depot A and symmetric distances: AB=10, AC=15, AD=20, BC=35, BD=25, CD=30. Enumerate all six customer permutations independently. The minimum tour is A–B–D–C–A with cost 10+25+30+15=80 (its reverse has the same cost). The exhaustive algorithm should return cost 80.

6.4 Local Search Alternative

Use 2-opt local search. Start with a random tour, then repeatedly choose two edges and reverse the segment between them if this reduces total distance. Stop when no improving 2-opt move exists. Run at least 20 independent trials using different random initial tours. For each trial, record best cost, execution time and percentage gap: ((local cost−optimal cost)/optimal cost)×100.

```
2_OPT(tour):
  improved = true
  while improved:
    improved = false
    for each pair of non-adjacent edges (i,i+1) and (j,j+1):
      candidate = reverse tour[i+1 ... j]
      if cost(candidate) < cost(tour):
        tour = candidate
        improved = true
  return tour
```

6.5 Inquiry and Deployment

Adding customers causes a dramatic increase because the number of possible tours grows factorially. Local search checks only nearby alternatives and is much faster on larger instances, but it can stop at a local minimum or plateau where no single 2-opt move improves the route even though a better global tour exists. A practical deployment policy is to use exhaustive search for very small n (for example, up to 10 customers as an initial engineering limit), then use 2-opt or another local-search method for larger n. The final boundary must be confirmed using measured response time on the target computer.

TASK 7 – Diagnose, Redesign and Validate (13 Marks)

Selected Solution: Delivery Tour Planning

The exhaustive delivery-tour algorithm is selected because its factorial growth creates a clear computational bottleneck as the number of customers increases.

A. Diagnose the Computational Bottleneck (2 Marks)

The main cost is generating and evaluating every permutation of customer visits. With the depot fixed, the number of tours is (n−1)!. Each extra customer multiplies the number of candidates by n−1. Most

candidate tours are evaluated independently even when they contain similar partial routes. This repeated work causes the limitation.

B. Define the Design Objective (2 Marks)

The redesign objective is to reduce the number of complete tours evaluated while returning a valid route quickly. For small instances where the exhaustive optimum is available, measure the percentage gap from the optimum. For all instances, record candidate tours or partial states examined, execution time and final route cost. The objective is to keep route quality close to the optimum while reducing computation.

C. Revised Computational Strategy (4 Marks)

Use branch-and-bound with a lower-bound estimate. Build a route one customer at a time. Maintain the best complete tour cost found so far (upper bound). For each partial route, calculate a lower bound on the cheapest possible completion. If the lower bound is greater than or equal to the best complete tour cost, stop exploring that branch because it cannot improve the current best solution. A simple safe lower bound can add the current path cost to the minimum outgoing connection required for each unvisited city, while avoiding double-counting; a stronger implementation can use a minimum spanning tree over unvisited cities plus connection costs from the current endpoint and back to the depot.

```
BRANCH_AND_BOUND(partial_route, unvisited, current_cost):
  if unvisited is empty:
    total = current_cost + distance(last(partial_route), depot)
    if total < best_cost:
      best_cost = total
      best_route = partial_route + [depot]
    return

  bound = current_cost + MST_cost(unvisited)
        + min_distance(last(partial_route), unvisited)
        + min_distance(unvisited, depot)

  if bound >= best_cost:
    return

  for city in unvisited ordered by distance from last(partial_route):
    BRANCH_AND_BOUND(partial_route + [city],
                     unvisited - {city},
                     current_cost + distance(last(partial_route), city))
```

The MST-based lower bound is admissible for symmetric non-negative distances because any completion must connect all unvisited cities and connect that set to the current endpoint and depot. Move ordering explores nearby cities first and may find a strong complete route early, tightening the upper bound. The redesign uses extra memory for the current route and best route, and extra computation for lower-bound calculations. It can still be exponential in the worst case, but it can prune many branches on suitable instances.

D. Predict Before Implementing (1 Mark)
The redesigned method is expected to examine fewer complete tours than exhaustive enumeration, especially when good routes are found early and the lower bound is strong. The primary expected improvement is the number of partial states and complete tours explored. Route quality should remain optimal if the lower bound is valid and pruning is implemented correctly.

E. Implement and Validate (2 Marks)
Run exhaustive search and branch-and-bound on the same distance matrices for small instances where exhaustive search can finish. Compare the returned route costs to confirm that both methods return the same optimum. Record nodes or partial states visited, complete tours evaluated, lower-bound calculations and execution time. Then test larger instances where exhaustive search is too slow, reporting branch-and-bound's best route and time without claiming an unverified optimum.

F. Defend the Final Design (2 Marks)
The redesign succeeds if it returns the same optimum on the small verification cases while evaluating fewer complete tours. The improvement depends on the quality of the lower bound and the order in which cities are explored. Its costs are extra lower-bound calculations and memory. On very small instances, exhaustive search may remain preferable because it is simpler and has little overhead. On difficult instances where the lower bound is weak, branch-and-bound may still explore nearly all permutations; local search can then be used when a fast approximate route is acceptable.

Conclusion
No single search method is best for every problem. A* is a strong default for informed grid routing, online search is useful when maps are unknown but less attractive for stable repeated missions, alpha-beta performance depends heavily on move ordering, minimax grows exponentially with depth, constraint propagation reduces CSP domains before or during backtracking, and local search provides a practical alternative to factorial exhaustive tour enumeration. All final performance claims should be supported by the student's own program outputs and raw operation counts.

Appendix – Suggested Experimental Record
For each experiment, record: test ID, input size, map/tree/problem description, random seed, algorithm, nodes expanded or visited, nodes generated, maximum frontier/recursion depth, number of cutoffs or constraint checks where relevant, solution cost, optimality status, execution time, and machine/software details. Repeat timing runs at least five times where appropriate; Task 6 requires at least 20 local-search runs.

Important: replace the example test plans and expected counts with actual program outputs before submission. The assignment explicitly requires source code, raw experimental data and tables/graphs, and states that operation counts are primary evidence.

Experimental Data Recording Requirements
For each experiment, record the following details: test ID, input size, map/tree/problem description, random seed, algorithm, nodes expanded or visited, nodes generated, maximum frontier/recursion depth,

number of cutoffs or constraint checks where relevant, solution cost, optimality status, execution time, and machine/software details.

Repeat timing runs at least five times where appropriate. For Task 6, perform at least 20 local-search runs.

Important: Replace the example test plans and expected counts with actual program outputs before submission. The assignment explicitly requires source code, raw experimental data and tables/graphs, and states that operation counts are primary evidence.

Experimental Results – Blank Recording Template

Test ID	Input size	Map/tree/problem description	Random seed	Algorithm	Nodes expanded/visited	Nodes generated	Max frontier/recursion depth	Cutoffs/constraint checks	Solution cost	Optimality status	Execution time	Machine/software details
T1-A	10×10 grid	Open grid; start (0,0), goal (9,9)	42	UCS / GBFS / A*	Record after run	Record after run	Record after run	N/A	Record after run	Check against shortest path	5 runs; record each	Record PC, OS, Python version
T1-B	10×10 grid	Cluttered grid with vertical wall and one opening	42	UCS / GBFS / A*	Record after run	Record after run	Record after run	N/A	Record after run	Check against shortest path	5 runs; record each	Record PC, OS, Python version
T1-C	10×10 grid	Dead-end corridor trap plus alternate route	42	UCS / GBFS / A*	Record after run	Record after run	Record after run	N/A	Record after run	Check against shortest path	5 runs; record each	Record PC, OS, Python version
T2-A	Small maze	Reachable target close to start; local	42	Online DFS	Record after run	N/A	Max stack depth	N/A	Moves travelled	Compare with full-map shorte	5 runs; record	Record PC, OS, Python version

		sensing only								st path	eac h	
T 2 - B	Me diu m ma ze	Reacha ble target far from start; modera te dead ends	42	Onli ne DFS / LRT A*	Recor d after run	N/ A	Max stack depth	N/A	Move s travell ed	Comp are with full- map shorte st path	5 run s; rec ord eac h	Record PC, OS, Python version
T 2 - C	Lar ge ma ze	Unreac hable target in disconn ected region	42	Onli ne DFS	Recor d after run	N/ A	Max stack depth	N/A	N/A – unrea chable	Repor t failur e/cov erage	5 run s; rec ord eac h	Record PC, OS, Python version
T 3 - A	De pth 4, bra nch ing 3	Game tree with favoura ble move orderin g	42	Mini max / Alph a- beta	Recor d after run	N/ A	Recurs ion depth 4	Record cutoffs	Recor d minim ax value	Value s must match	5 run s; rec ord eac h	Record PC, OS, Python version
T 3 - B	De pth 4, bra nch ing 3	Game tree with deliber ately poor move orderin g	42	Mini max / Alph a- beta	Recor d after run	N/ A	Recurs ion depth 4	Record cutoffs	Recor d minim ax value	Value s must match	5 run s; rec ord eac h	Record PC, OS, Python version
T 3 - C	De pth 4, bra nch ing 3	Equal leaf values or shuffle d child order	42	Mini max / Alph a- beta	Recor d after run	N/ A	Recurs ion depth 4	Record cutoffs	Recor d minim ax value	Value s must match	5 run s; rec ord eac h	Record PC, OS, Python version
T 4 - A	b=3 , d=2	Uniform game tree; depth- limited	42	Mini max	Recor d after run	N/ A	Recurs ion depth 2	N/A	Best move/ evalua tion	Check expec ted 9 leaves	5 run s; rec ord eac h	Record PC, OS, Python version

		evaluation										
T4-B	b=3, d=4	Uniform game tree; depth-limited evaluation	42	Minimax	Record after run	N/A	Recursion depth 4	N/A	Best move/ evaluation	Check expected 81 leaves	5 runs; record each	Record PC, OS, Python version
T4-C	b=4, d=4	Larger branching factor test	42	Minimax	Record after run	N/A	Recursion depth 4	N/A	Best move/ evaluation	Check expected 256 leaves	5 runs; record each	Record PC, OS, Python version
T5-A	3 exams	Pairwise conflicts; restricted domains for propagation-only success	42	Backtracking + AC-3	Record after run	N/A	Max recursion depth	Constraint checks	Timetable assignment	Valid if all constraints hold	5 runs; record each	Record PC, OS, Python version
T5-B	2 exams	Two conflicting exams both restricted to slot {1}	42	Backtracking + AC-3	Record after run	N/A	Max recursion depth	Constraint checks/ domain wipe-outs	No solution	Must detect empty domain	5 runs; record each	Record PC, OS, Python version

The rows below are populated experiment plans, not measured results. Cells marked 'Record after run' must be replaced with actual output from your implementation. Do not submit planned values as experimental evidence.